

## SET6. A2. Кратчайший блиц!

### 1. Модификация алгоритма Дейкстры для произведения весов

**Модифицированный алгоритм DijkstraMULT(G, start):**

1. Стартовой вершине присваиваем значение 1, всем остальным вершинам присваиваем бесконечность.
2. Кладем все вершины в приоритетную очередь, где самой важной считается вершина с самым маленьким текущим значением.
3. Пока в очереди есть вершины: берем из очереди вершину с самым маленьким текущим значением и «закрепляем» ее (больше ее значение не меняем); для каждой вершины, в которую ведет ребро из этой закрепленной вершины:
  - Считаем новое возможное значение = (текущее значение закрепленной вершины)  $\times$  (вес ребра).
  - Если это новое значение меньше того, что сейчас записано для соседней вершины, то обновляем ее значение и заново кладем ее в очередь с новым (меньшим) приоритетом.

Алгоритм работает правильно только для графов, где все веса ребер больше или равны 1, т.к. когда все веса  $\geq 1$ , произведение весов вдоль пути никогда не уменьшается при добавлении новых ребер. Поэтому, как только мы закрепили вершину, ее значение уже точно минимальное — через еще не обработанные вершины ничего лучше не получится.

**Контрпример**, если есть вес меньше 1:

Вершины:  $s \rightarrow u$  (вес 10),  $u \rightarrow t$  (вес 0.1),  $s \rightarrow t$  (вес 2).

Настоящий лучший путь  $s \rightarrow t$  дает произведение 1.

Алгоритм сначала закрепит  $t$  со значением 2, а потом через  $u$  уже не сможет обновить его, следовательно ответ будет ошибочным (2 вместо 1).

### 2. Алгоритм RestoreGraph(dist[1..n][1..n])

**Псевдокод:**

```
if  $\exists i: \text{dist}[i][i] \neq 0$  return «impossible» // Отрицательный цикл
```

```
for  $\forall i, j, k$ :
```

```
    if  $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ : return «impossible» // Нарушение неравенства треугольника
```

```
adj = [ ]
```

```
for  $u \neq v$ :
```

```
    if  $\text{dist}[u][v] < \text{inf}$ :
```

```
        // Проверяем, обязателен ли прямой ребро
```

```
        is_direct = true:
```

```
        for  $k = 1$  to  $n$ :
```

```
            if  $k \neq u$  and  $k \neq v$ :
```

```
                if  $\text{dist}[u][k] + \text{dist}[k][v] == \text{dist}[u][v]$ : is_direct = false; break
```

```
        if is_direct:
```

```
            adj[u].add((v, dist[u][v]))
```

```
return adj
```

### Случай невозможности восстановления:

- $\text{dist}[i][i] \neq 0$  (отрицательный цикл)
- Нарушено неравенство треугольника (ни один граф не может давать такую матрицу)

**Пояснение:** Если для каких-то  $i, j, k$  выполняется:  $\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j]$ , то путь через  $k$  короче, чем «кратчайший» путь из матрицы → противоречие. Такая матрица не может быть матрицей кратчайших расстояний ни в каком графе → восстановление невозможно. Аналогично:  $\text{dist}[i][i] \neq 0$  — либо отрицательный цикл, либо ошибка в матрице.

## 3. Поиск ошибка в ядре Флойда-Уоршелла

### Ошибка в порядке вложенности циклов:

В правильной реализации алгоритма Флойда–Уоршелла внешним должен быть цикл по промежуточной вершине  $k$ , а внутренними — по  $i$  и  $j$ . В приведенном коде  $k$  находится во внутреннем цикле, поэтому обновления расстояний не успевают обновиться за один проход по всем вершинам.

**Пример графа (4 вершины):**  $1 \rightarrow 4$  (1),  $4 \rightarrow 3$  (1),  $3 \rightarrow 2$  (1),  $1 \rightarrow 2$  (10)

Кратчайший путь  $1 \rightarrow 2$ :  $1 \rightarrow 4 \rightarrow 3 \rightarrow 2$ , длина 3.

Выполнение ошибочного кода (только  $i = 1$ ):

1. Изначально:  $\text{dist}[1][2] = 10$ ,  $\text{dist}[1][3] = \text{inf}$ ,  $\text{dist}[1][4] = 1$ ,  $\text{dist}[3][2] = 1$ ,  $\text{dist}[4][3] = 1$
2. При  $i = 1, j = 2$ : все  $k$  дают  $\geq 10 \rightarrow \text{dist}[1][2]$  остается 10
3. При  $i = 1, j = 3$ : через  $k = 4 \rightarrow \text{dist}[1][3] = 2$  (обновилось)
4.  $j = 2$  уже обработан  $\rightarrow$  итог  $\text{dist}[1][2] = 10$  (ошибка) ч.т.д.

## 4. Ответ: Да

### Пример:

Ребра:  $1 \rightarrow 2$ (1),  $2 \rightarrow 3$ (1),  $3 \rightarrow 4$ (1),  $4 \rightarrow 2$ (10),  $3 \rightarrow 1$ (10)

Кратчайший  $1 \rightarrow 4$ :  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$  (длина 3), ребро  $(2 \rightarrow 3)$  лежит на нем

Кратчайший  $4 \rightarrow 1$ :  $4 \rightarrow 2 \rightarrow 3 \rightarrow 1$  (длина 21), то же ребро  $(2 \rightarrow 3)$  лежит на нем

### Пояснение:

- Существует направленный цикл положительного веса:  $2 \rightarrow 3 \rightarrow 1 \rightarrow 2$  (или через 4). Вес цикла  $> 0 \rightarrow$  нет отрицательного цикла  $\rightarrow$  все стандартные алгоритмы поиска кратчайших путей работают корректно.
- Ребро  $(2 \rightarrow 3)$  лежит на обоих направлениях, потому что оно является выгодным переходом в обоих направлениях в данном графе.
- Кратчайшие пути остаются простыми (не содержат циклов), потому что все веса положительные  $\rightarrow$  добавление цикла только увеличивает длину.

**Ограничения применимости известных алгоритмов:** Граф не содержит отрицательных циклов, поэтому все стандартные алгоритмы поиска кратчайших путей (Дейкстра, Беллман–Форд, Флойд–Уоршелл) применимы без ограничений.